Universitatea Tehnică din Cluj-Napoca

Facultatea de Calculatoare si Tehnologia Informatiei

Sistem de Control pentru Servo Motor folosind FPGA și PID

- Raport Proiect -

Student: Purenciu Diana
Grupa: 30236

Îndrumător proiect:
Prof. Mircea Paul Muresan

Cluj-Napoca
22 ianuarie 2026

Cuprins

1 Rezumat **2**

2 Introducere **3**
2.1 Contextul și importanța domeniului . . . 3
2.2 Definirea problemei și obiectivele proiectului . . . 3
2.3 Soluția propusă . . . 4

3 Fundamentare teoretică **5**
3.1 Principiul de funcționare al Servomotoarelor . . . 5
3.2 Arhitectura Sistemelor Hibride . . . 5
3.3 Aritmetica Digitală: Virgula Fixă vs. Virgula Mobilă . . . 6
3.4 Conversia Analog-Digitală . . . 6

4 Proiectare și Implementare **7**
4.1 Metodologia și Soluția Aleasă . . . 7
4.2 Arhitectura Generală a Sistemului . . . 7
4.3 Implementarea Modulelor de Procesare Numerică . . . 8
4.3.1 1. Modulul de Conversie a Datelor (FP Converter) . . . 8
4.3.2 2. Modulul de Scalare Hardware (Conv PWM Int) . . . 8
4.3.3 3. Unitatea Aritmetică în Virgulă Mobilă (FPU) . . . 9
4.3.4 4. Algoritmul de Control (PID Core) . . . 9
4.3.5 5. Unitatea Centrală de Comandă (Top-Level Controller) . . . 10
4.3.6 6. Generatorul de Semnal PWM (Servo Driver) . . . 10
4.4 Detalii de Implementare Software (PS) . . . 10
4.4.1 1. Comunicația PS-PL prin AXI GPIO . . . 10
4.4.2 2. Configurarea Modulului de Achiziție (XADC) . . . 10
4.4.3 3. Fluxul de Execuție (Bucla Principală) . . . 11

5 Rezultate Experimentale și Validarea Designului **12**
5.1 Validarea prin Simulare . . . 12
5.1.1 1. Validarea Unităților Aritmetice (FPU) . . . 12
5.1.2 2. Verificarea Logicii PID . . . 13
5.2 Adaptarea Sistemului de Vizualizare . . . 14
5.2.1 1. Strategia de Vizualizare Hibridă . . . 14
5.2.2 2. Implementarea Hardware (Blocul xlconcat) . . . 14

6 Concluzii **15**
6.1 Contribuții Personale . . . 15
6.2 Avantaje și Dezavantaje . . . 15
6.3 Aplicații și Dezvoltări Viitoare . . . 15

Anexe **18**

1 Rezumat

Acest proiect a avut ca scop realizarea unui sistem de control pentru un servomotor, utilizând placa de dezvoltare Zybo Z7. Obiectivul principal a fost controlul precis al poziției motorului în funcție de valoarea citită de la un potențiometru. Pentru a rezolva această problemă, am folosit o metodă combinată, care împarte sarcinile între partea hardware și cea software, folosind mediile de lucru Vivado și Vitis. Semnalul de comandă pentru motor (PWM), care necesită o precizie mare de timp (impulsuri între 1ms și 2ms), precum și calculele matematice necesare pentru ajustarea duratei acestuia, au fost realizate direct în hardware folosind limbajul VHDL. Citirea datelor de la senzori și transmiterea lor către sistem au fost realizate în software, prin cod scris în limbajul C. Rezultatele experimentale au demonstrat funcționarea corectă a sistemului, lucru vizibil prin aprinderea unui LED controlat hardware proporțional cu comanda, simultan cu afișarea nivelului de intensitate pe alte trei LED-uri controlate din software. În concluzie, proiectul a arătat că utilizarea împreună a logicii hardware și a procesării software reprezintă o soluție eficientă și flexibilă pentru controlul motoarelor.

2 Introducere

În prezent, domeniul automatizărilor și al roboticii trece printr-o perioadă de dezvoltare accelerată, influențând majoritatea sectoarelor industriale. De la brațe robotice industriale care asamblează mașini cu o precizie milimetrică, până la drone de livrare și vehicule autonome, toate aceste sisteme complexe au nevoie de o componentă esențială: elemente de execuție precise și fiabile. Cel mai răspândit element de acest tip este servomotorul, un dispozitiv electromecanic capabil să își mențină poziția la un unghi exact în funcție de comanda primită, oferind un raport excelent între putere și dimensiuni [1].

Pentru ca aceste motoare să funcționeze corect și fluid, ele au nevoie de comenzi extrem de rapide și stabile. O mică eroare sau întârziere în semnalul de comandă poate duce la vibrații nedorite, consum inutil de energie sau chiar la poziționarea greșită a robotului, cu potențiale consecințe grave. Acest proiect abordează problema controlului precis al unui servomotor folosind tehnologii moderne de tip FPGA (Field Programmable Gate Array), propunând o alternativă superioară metodelor clasice bazate strict pe software.

2.1 Contextul și importanța domeniului

Sistemele de calcul moderne nu mai sunt limitate la calculatoarele clasice de pe birou sau laptopuri. Tot mai des întâlnim conceptul de "Sisteme Embedded" (sisteme încorporate), unde un procesor și circuite logice lucrează împreună "invizibil" pentru a controla un echipament, fie că este vorba despre un aparat medical, un sistem de frânare auto sau o imprimantă 3D [2].

În contextul actual, există o tendință clară în industrie de a muta sarcinile care necesită viteză foarte mare dinspre software (programe clasice rulate secvențial) înspre hardware (circuite dedicate care funcționează paralel). Această abordare, cunoscută sub numele de accelerare hardware, este esențială în robotică, deoarece permite procesorului să se ocupe de decizii complexe (inteligență artificială, comunicare, interfața cu utilizatorul), în timp ce hardware-ul se ocupă de mișcarea efectivă a motoarelor. Astfel, se obține un sistem mult mai reactiv și mai robust.

Acest proiect se încadrează în domeniul actual al proiectării hardware-software (co-design), utilizând o platformă de tip System-on-Chip (SoC). Aceasta combină flexibilitatea unui procesor cu viteza brută a circuitelor logice programabile, fiind soluția ideală pentru aplicații de timp real unde fiecare microsecundă contează [3].

2.2 Definirea problemei și obiectivele proiectului

Problema principală pe care acest proiect dorește să o rezolve este limitarea sistemelor de control realizate exclusiv prin software. Atunci când un procesor obișnuit (microcontroller) trebuie să genereze semnale foarte rapide, cum este semnalul PWM pentru un servomotor, pot apărea întârzieri neprevăzute. Dacă procesorul este ocupat cu alte calcule, gestionarea memoriei sau tratarea unor întreruperi de sistem, semnalul către motor poate întârzia câteva microsecunde. Această variație nedorită a timpului, numită tehnic "jitter", face ca motorul să tremure sau să nu stea fix în poziție, reducând precizia sistemului.

Obiectivul general al proiectului este realizarea unui sistem de control stabil, robust și precis, care elimină aceste erori prin utilizarea tehnologiei FPGA pentru generarea semnalelor critice.

Obiectivele specifice urmărite sunt:

- Implementarea unui circuit digital în limbajul VHDL care să genereze semnalul PWM cu o precizie ridicată, respectând cu strictețe timpii standard (impulsuri între 1ms și 2ms), independent de încărcarea procesorului.
- Realizarea calculelor matematice necesare pentru conversia comenzii în timp direct în hardware, pentru a descărca procesorul de această sarcină computațională.
- Integrarea circuitului hardware cu un procesor, folosind interfețe standardizate de tip AXI, pentru a permite controlul și monitorizarea sistemului dintr-o aplicație software.
- Preluarea unei comenzi externe analogice (potențiometru) prin convertorul XADC și afișarea stării sistemului pe LED-uri, oferind un feedback vizual imediat utilizatorului.

2.3 Soluția propusă

Soluția aleasă pentru acest proiect este una hibridă, implementată pe placa de dezvoltare Zybo Z7. Spre deosebire de soluțiile clasice care folosesc doar un microprocesor (cum ar fi Arduino sau Raspberry Pi), soluția noastră separă clar responsabilitățile pe două niveluri:

1. **Partea Hardware (VHDL):** Ea primește doar valoarea finală dorită și se ocupă singură de numărarea timpului și generarea impulsurilor electrice. Această parte nu poate fi întreruptă de nimic, deci semnalul este perfect stabil și repetitiv.
2. **Partea Software (C):** Ea citește senzorii, interpretează datele, decide ce trebuie să facă sistemul și trimite comanda către hardware.

Această arhitectură este superioară variantelor pur software deoarece garantează că, indiferent cât de ocupat este procesorul cu alte sarcini, motorul va primi mereu semnalul corect. Mai mult, această soluție este scalabilă: același principiu ar putea fi folosit pentru a controla zeci de motoare simultan fără a îngreuna procesorul.

3 Fundamentare teoretică

Acest capitol prezintă tehnologiile și conceptele fundamentale utilizate în realizarea proiectului. S-a pus accent pe înțelegerea mecanismelor de control digital, a standardelor de comunicare cu elementele de execuție și a arhitecturii specifice a sistemelor hibride FPGA.

3.1 Principiul de funcționare al Servomotoarelor

Servomotorul este un sistem de acționare în buclă închisă, compus dintr-un motor de curent continuu, un reductor mecanic și un circuit electronic de control integrat. Spre deosebire de motoarele convenționale, servomotorul nu este controlat prin variația tensiunii de alimentare, ci printr-un semnal digital modulat în timp [1].

Metoda de control utilizată este PWM (Modulația în durata impulsurilor). Este important de reținut diferența față de motoarele obișnuite:

- La un motor simplu, PWM-ul controlează viteza (cu cât impulsul e mai lung, cu atât motorul primește mai multă energie și se învârte mai tare).
- La un servomotor, PWM-ul controlează poziția. Lungimea exactă a timpului cât semnalul este activ îi spune motorului la ce unghi să se oprească.

Pentru a funcționa corect, servomotoarele standard (RC Servo) necesită respectarea următorilor timpi [1]:

- **Perioada de repetiție (T):** Semnalul trebuie să aibă o frecvență fixă de 50 Hz, ceea ce corespunde unei perioade de 20 ms.
- **Durata impulsului (t_{on}):** Aceasta dictează unghiul axului.
  - 1.0 ms → Poziția -90 grade (Stânga maxim).
  - 1.5 ms → Poziția 0 grade (Centru / Neutru).
  - 2.0 ms → Poziția +90 grade (Dreapta maxim).

3.2 Arhitectura Sistemelor Hibride

Pentru realizarea proiectului s-a utilizat o placă de dezvoltare bazată pe o arhitectură hibridă. Această platformă este ideală pentru aplicații complexe deoarece integrează într-o singură unitate două componente esențiale [2]:

1. **Processing System (PS):** Partea de procesare software care oferă flexibilitate, permițând rularea de sisteme de operare sau cod C complex pentru interfețe și calcule la virgulă mobilă.
2. **Programmable Logic (PL):** Partea de hardware reconfigurabil (FPGA). Aceasta oferă paralelism, fiind ideală pentru sarcini critice de timp, cum ar fi generarea semnalelor PWM sau achiziția rapidă de date.

Un avantaj major al acestei arhitecturi este legătura rapidă dintre procesor și partea hardware. Comunicarea se realizează printr-o interfață standard numită AXI. În proiectul nostru, am folosit un modul specific (AXI GPIO) care funcționează ca o punte de legătură. Acesta permite procesorului să trimită comenzile către circuitul hardware instantaneu și să citească starea acestuia, asigurând o colaborare eficientă între cele două părți.

3.3 Aritmetica Digitală: Virgula Fixă vs. Virgula Mobilă

În implementarea algoritmilor de control pe FPGA, precizia calculelor depinde fundamental de modul de reprezentare a numerelor. Există două abordări teoretice distincte [5]:

- **Virgula Fixă (Fixed-Point):** Este metoda standard în proiectarea hardware, similară lucrului cu numere întregi. Deși este extrem de rapidă și eficientă ca resurse, prezintă dezavantajul unei precizii limitate. În calculele iterative, erorile de trunchiere se pot acumula, afectând stabilitatea sistemului.
- **Virgula Mobilă (Floating-Point - Standardul IEEE 754):** Aceasta este metoda utilizată implicit în procesoarele de PC, permițând reprezentarea numerelor foarte mari sau foarte mici prin notația științifică ($mantisă \times 2^{exponent}$).

Din punct de vedere teoretic, implementarea operatorilor matematici în virgulă mobilă direct în structura hardware a FPGA-ului este o sarcină complexă, care necesită circuite logice pentru alinierea și normalizarea biților [6]. Totuși, acest efort este justificat în sistemele de control avansate, deoarece oferă o precizie matematică superioară.

3.4 Conversia Analog-Digitală

Pentru stabilirea referinței de poziție, se utilizează un potențiometru. Acesta funcționează pe principiul divizorului de tensiune. Deoarece FPGA-ul operează în regim digital, tensiunea analogică este transformată într-o valoare numerică prin intermediul modulului intern **XADC** [7].

Acesta dispune de o rezoluție de 12 biți, divizând intervalul de tensiune de intrare (0V - 1V) în 4096 de trepte. Astfel, poziția fizică a potențiometrului este convertită într-o valoare numerică cuprinsă între 0 și 4095, utilizată ulterior de sistem ca referință.

4 Proiectare și Implementare

Acest capitol detaliază procesul tehnic de realizare a sistemului, pornind de la alegerea metodei de implementare, descrierea arhitecturii hardware-software și până la algoritmii utilizați pentru controlul servomotorului.

4.1 Metodologia și Soluția Aleasă

Pentru atingerea obiectivelor proiectului, s-a optat pentru o implementare hibridă de tip Hardware-Software. Această abordare exploatează arhitectura duală a cipului, distribuind sarcinile în mod optim pentru performanță:

- **Logica Programabilă (PL):** Funcționează ca un accelerator hardware. Aici sunt implementate atât generarea semnalului PWM, cât și unitățile de calcul matematic în virgulă mobilă. Executarea algoritmilor direct în hardware asigură o viteză de procesare superioară.
- **Sistemul de Procesare (PS):** Acesta gestionează fluxul de date și interfața, dar este eliberat de sarcina intensivă a calculelor matematice, care sunt descărcate ("offloaded") către PL.

Alternative analizate și abandonate: În faza inițială de proiectare, au fost considerate următoarea variantă: **Soluție exclusiv Software:** Deși mai simplă, a fost respinsă din două motive: generarea PWM software induce fluctuații, iar calculele complexe în virgulă mobilă ar fi consumat timp semnificativ de procesor, crescând latența.

4.2 Arhitectura Generală a Sistemului

Sistemul este organizat sub forma unei bucle de control accelerată hardware. Fluxul de date este conceput astfel încât procesorul să gestioneze comunicația, iar logica programabilă să execute calculele intensive. Schema bloc funcțională cuprinde trei module majore, interconectate prin magistrala AXI:

1. **Modulul de Achiziție (XADC):** Interfața hardware care preia tensiunea de la potențiometru și o convertește în format digital.
2. **Unitatea de Management (Software - PS):** Rolul său este de a citi valoarea brută de la XADC și de a o transmite direct către acceleratorul din PL, fără a o procesa.
3. **Acceleratorul Matematic și de Execuție (Hardware - PL):** Este componenta centrală care preia toate sarcinile complexe. Arhitectura sa internă este structurată pe trei etaje:
    - **Conversia Datelor:** Transformă numerele întregi în format virgulă mobilă (Float) pentru precizie și invers, pentru execuție.
    - **Unitatea Aritmetică:** Execută operațiile matematice necesare algoritmului de control (PID/Scalare) folosind module hardware dedicate pentru adunare, scădere, înmulțire și împărțire în virgulă mobilă.

- **Generarea Semnalului:** Rezultatul calculului este transformat într-un semnal electric de comandă. Modulul generează impulsuri cu o durată precisă, cuprinsă strict între 1 ms și 2 ms, asigurând poziționarea corectă a servomotorului.

4.3 Implementarea Modulelor de Procesare Numerică

4.3.1 1. Modulul de Conversie a Datelor (FP Converter)

Entitatea `fp_converter` funcționează ca o punte bidirecțională între domeniul numerelor întregi și standardul IEEE 754. Arhitectura sa se bazează pe o mașină de stare finită (FSM) care gestionează două procese distincte:

A. Conversia Intreg → Virgulă Mobilă (I2F) Deoarece ADC-ul furnizează o valoare brută pe 12 biți, aceasta trebuie normalizată. Algoritmul parcurge următorii pași:

1. **Detectarea MSB (Most Significant Bit):** Se scanează numărul de intrare de la stânga la dreapta pentru a găsi prima poziție de "1". Aceasta determină exponentul brut.

2. **Calculul Exponentului:** Se aplică formula standard IEEE 754:

$$Exponent = Bias(127) + Pozitie_{MSB}$$

3. **Alinierea Mantisei:** Valoarea întreagă este shiftată la stânga astfel încât cel mai semnificativ bit să se alinieze cu bitul 23 al formatului virgulă mobilă, ascunzând bitul implicit.

B. Conversia Virgulă Mobilă → Intreg (F2I) După calculul PID, rezultatul trebuie transformat înapoi într-un număr întreg pentru a comanda motorul.

- **Extragerea Exponentului:** Se elimină bias-ul de 127 pentru a afla puterea reală a lui 2.

- **Shiftarea Mantisei:** Se adaugă bitul implicit "1" la mantisă și se shiftează valoarea la dreapta conform exponentului calculat.

- **Mecanismul de Clamping (Saturație):** Dacă rezultatul depășește valoarea maximă reprezentabilă pe 12 biți (4095), ieșirea este forțată la valoarea maximă (`MAX_12_BIT`), prevenind erorile de tip *overflow*.

4.3.2 2. Modulul de Scalare Hardware (Conv PWM Int)

Entitatea `conv_pwm_int` realizează adaptarea domeniului numeric. Deoarece valoarea maximă a ADC-ului este 4095, iar rezoluția PWM dorită este de 8 biți (0-255), este necesară o operație de scalare conform formulei:

$$PWM_{out} = \frac{Val_{in} \times 255}{4095}$$

Formula este descompusă în două operații primitive hardware:

Înmulțitorul Secvențial (Shift-and-Add Algorithm) Modulul implementează algoritmul clasic de multiplicare binară [5]. Mașina de stare verifică bitul cel mai puțin semnificativ (LSB) al multiplicatorului. Dacă acesta este "1", adună multiplicandul la produsul parțial. Procesul se repetă pentru toți biții.

Divizorul cu Restaurare (Restoring Division) S-a utilizat algoritmul de împărțire cu restaurare a restului [5].

- **Algoritmul pas cu pas:**
  1. Se shiftează restul parțial la stânga.
  2. Se scade divizorul din restul parțial curent.
  3. **Decizie și Restaurare:** Dacă rezultatul scăderii este negativ, bitul curent al câtului devine "0", iar restul este "restaurat" la valoarea dinaintea scăderii. Dacă rezultatul este pozitiv, bitul câtului devine "1".

4.3.3 3. Unitatea Aritmetică în Virgulă Mobilă (FPU)

Pentru realizarea algoritmului PID s-a implementat un set de operații aritmetice în standardul **IEEE 754 Single Precision (32 biți)** [6].

A. Sumatorul/Scăzătorul (vm_add_sub) Acest modul este cel mai complex, deoarece operanzii au exponenți diferiți. Mașina de stare controlează etapele secvențiale: extragerea datelor, alinierea mantiselor (shiftare în funcție de diferența exponenților), calculul aritmetic și normalizarea rezultatului.

B. Înmulțitorul (vm_mul) Operația de înmulțire este mai directă. Semnul final este determinat prin XOR, exponentul prin suma exponenților minus Bias, iar mantisele sunt înmulțite similar numerelor întregi, rezultatul fiind trunchiat la 24 de biți.

C. Divizorul (vm_div) Acest modul este esențial pentru calculul termenului derivativ. Structura sa include mecanisme de protecție, precum detectarea împărțirii la zero, caz în care returnează o valoare specială (Infinit) pentru a evita blocarea sistemului.

4.3.4 4. Algoritmul de Control (PID Core)

Entitatea `pid_core` coordonează unitățile de calcul pentru a implementa ecuația discretă a regulatorului PID [8]:

$$u[n] = K_p \cdot e[n] + K_i \cdot \sum(e[n] \cdot T_s) + K_d \cdot \frac{e[n] - e[n-1]}{T_s}$$

Mașina de stare parcurge pașii elementari: calculul erorii, calculul celor trei termeni (Proporțional, Integrator - prin cumulare, Derivativ - prin diferența față de pasul anterior) și asamblarea ieșirii finale.

4.3.5 5. Unitatea Centrală de Comandă (Top-Level Controller)

Modulul `pid_controller` funcționează ca un manager al întregului sistem. Deoarece procesarea datelor necesită timp, modulul utilizează o mașină de stare pentru a sincroniza operațiile într-o ordine strictă, formând un flux de prelucrare (pipeline):

- **Starea IDLE:** Sistemul așteaptă semnalul de start.
- **Starea I2F:** Comandă convertorul int-float.
- **Starea PID:** Activează nucleul PID și așteaptă finalizarea calculului.
- **Starea F2I:** Rezultatul PID este reconvertit în întreg.
- **Starea PWM:** Valoarea întreagă este trimisă către generatorul de semnal.

4.3.6 6. Generatorul de Semnal PWM (Servo Driver)

Modulul `servo_pwm_gen` transformă numărul primit de la controler (0-255) în impulsuri electrice. Funcționarea se bazează pe ceasul de 125 MHz și urmează trei pași logici:

- **Controlul Frecvenței:** Un numărător măsoară timpul până ajunge la valoarea corespunzătoare perioadei de 20 ms.
- **Calculul Impulsului:** Durata activă ("1") este determinată dinamic, pornind de la 1 ms (bază) și adăugând timp proporțional cu intrarea.
- **Generarea Ieșirii:** Un comparator menține ieșirea activă cât timp numărătorul este sub pragul calculat.

4.4 Detalii de Implementare Software (PS)

Software-ul a fost dezvoltat în C, utilizând Vitis, și rulează pe procesorul ARM Cortex-A9. Acesta funcționează ca un dispecer de date.

4.4.1 1. Comunicația PS-PL prin AXI GPIO

Transferul de date se realizează prin magistrala AXI GPIO. Au fost definite canale pentru:

- **Semnale de Control:** Start, Reset.
- **Transfer de Date:** Potențiometru (intrare), PWM (ieșire).
- **Sincronizare:** Semnalul `done` de la hardware.

4.4.2 2. Configurarea Modulului de Achiziție (XADC)

Integrarea senzorului s-a realizat utilizând IP-ul `XADC Wizard`.

1. Configurare Generală S-a selectat interfața `AXI4Lite` și modul `Continuous Mode` cu `Channel Sequencer` activat pentru scanare automată.

2. Selecția Canalelor Au fost activate canalele auxiliare (`vaux6, vaux7, vaux14, vaux15`) corespunzătoare pinilor fizici ai plăcii Zybo.

3. Maparea Pinilor Porturile logice au fost declarate externe și mapate în fișierul de constrângeri (`.xdc`) pe pinii fizici (ex: Header JA).

4.4.3 3. Fluxul de Execuție (Bucla Principală)

Programul principal utilizează o buclă de tip "Polling":

- Citește valoarea de la XADC.
- Trimite valoarea către FPGA și ridică semnalul `Start`.
- Așteaptă într-o buclă `do-while` până când hardware-ul răspunde cu `Done`.
- Citește rezultatul calculat și îl trimite pe serială pentru monitorizare.

5 Rezultate Experimentale și Validarea Designului

Deoarece sistemul propus are un grad ridicat de complexitate, strategia de validare a fost împărțită în două etape majore: verificarea corectitudinii matematice prin simulare (pentru unitățile de calcul) și integrarea arhitecturală în mediul Vivado (pentru pregătirea testării fizice).

5.1 Validarea prin Simulare

Aceasta a reprezentat principala metodă de verificare a algoritmilor. S-au rulat scenarii de test (Testbenches) pentru a confirma că mașinile de stare funcționează conform specificațiilor.

5.1.1 1. Validarea Unităților Aritmetice (FPU)

Modulele de calcul în virgulă mobilă (`vm_add_sub`, `vm_mul`) au fost testate individual.

- **Obiectiv:** Confirmarea respectării standardului IEEE 754.
- **Rezultat (Adunare):** În Figura 1, s-a testat adunarea 1.0+2.0. Rezultatul obținut (3.0, Hex: `0x40400000`) confirmă alinierea corectă a mantiselor.

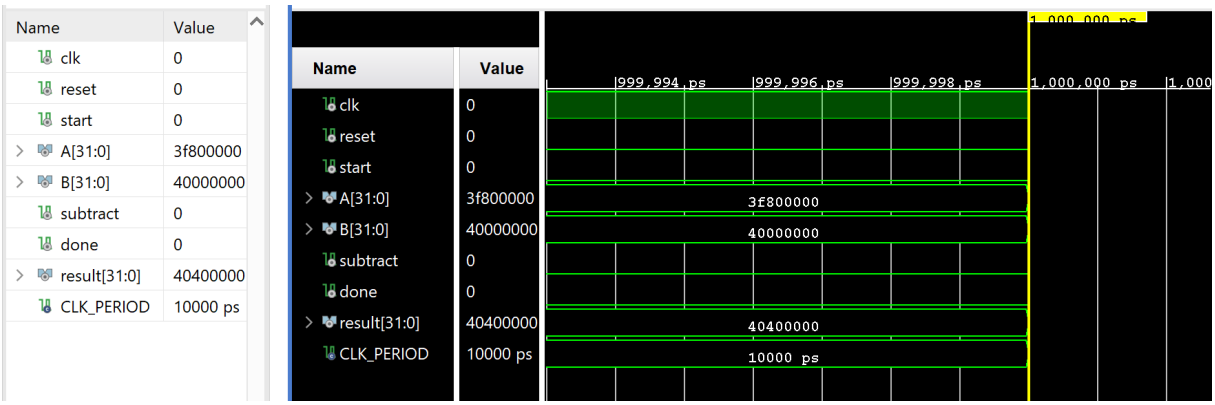

Figura 1: Cronograma simulării pentru unitatea aritmetică (Adunare). Se observă succesiunea stărilor și finalizarea calculului.

- **Rezultat (Înmulțire și Divizare):** În Figura 2, s-a verificat precizia înmulțirii ($0.5 \times 0.5 = 0.25$). Figura 3 demonstrează robustețea sistemului: la o împărțire cu zero (1.0/0.0), modulul a returnat corect codul pentru **Infinit** (Hex: `0x7F800000`).

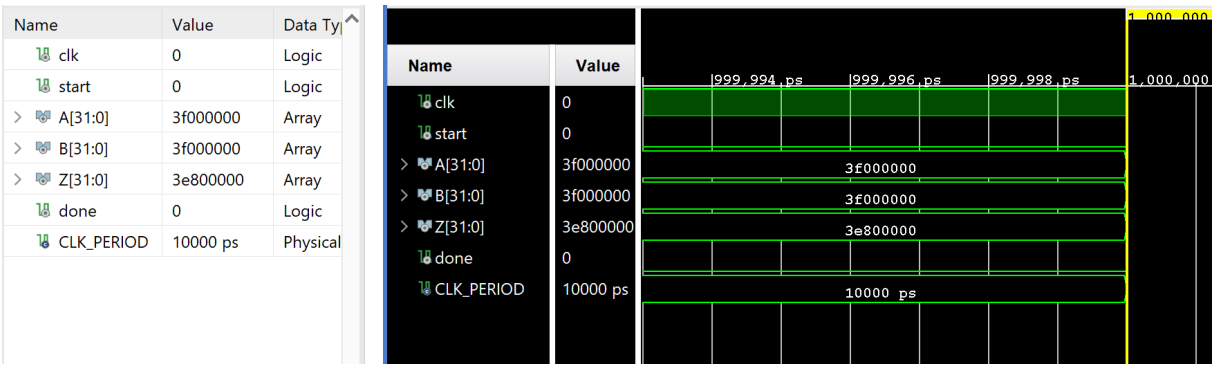

Figura 2: Simularea operației de înmulțire. Rezultatul 0.25 este generat corect.

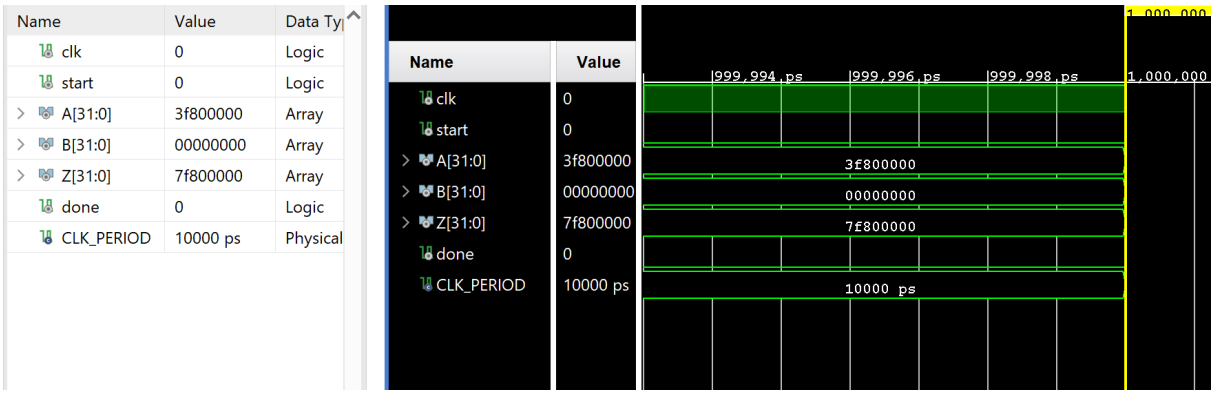

Figura 3: Simularea împărțirii. Se observă tratarea excepției "Divide by Zero" prin returnarea valorii Infinit.

5.1.2 2. Verificarea Logicii PID

S-a simulat funcționarea nucleului `pid_core` (Figura 4) aplicând o treaptă de referință în buclă deschisă.

- **Funcționalitate:** Simularea confirmă că mașina de stare parcurge corect etapele algoritmului.
- **Observație privind parametrii:** Deoarece simularea este realizată fără un motor real care să reducă eroarea, aceasta rămâne constantă. Acest lucru cauzează acumularea continuă a termenului Integrator (fenomenul de Wind-up), ducând rapid ieșirea către saturație. Acest comportament este corect teoretic, confirmând funcționalitatea matematică.

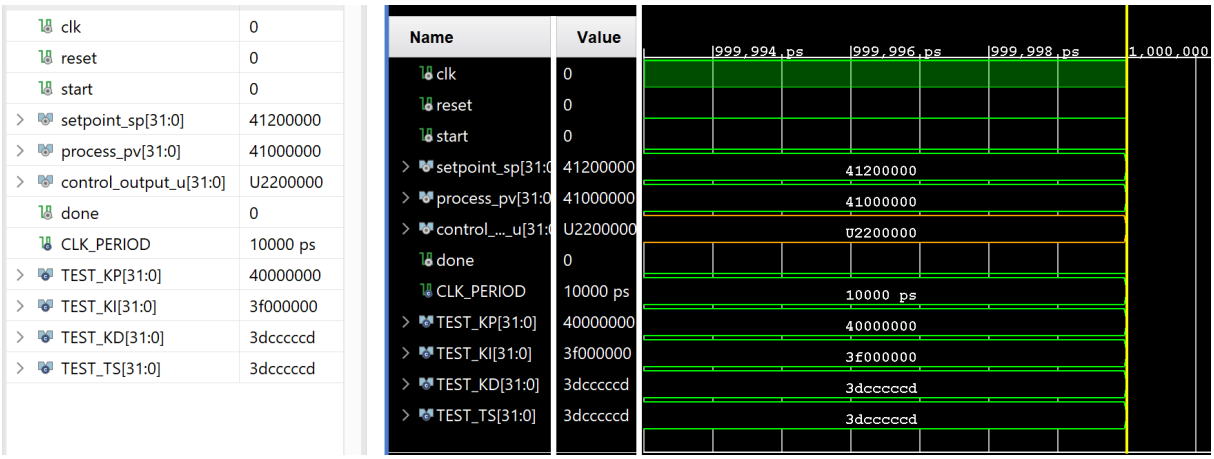

Figura 4: Simularea controlerului PID. Saturarea rapidă a ieșirii confirmă activitatea termenului integrator în condiții de eroare constantă.

5.2 Adaptarea Sistemului de Vizualizare

În vederea validării funcționale pe placa de dezvoltare, în absența sarcinii mecanice (servomotorul), interfața de ieșire a fost adaptată pentru a oferi un feedback vizual imediat operatorului.

5.2.1 1. Strategia de Vizualizare Hibridă

S-a implementat un sistem dual de monitorizare utilizând cele 4 LED-uri disponibile pe placa Zybo:

- **Vizualizarea Comenzii (PWM):** Primul LED este conectat direct la ieșirea generatorului PWM. Variația factorului de umplere modifică intensitatea luminoasă a acestuia, indicând puterea semnalului de comandă.
- **Vizualizarea Nivelului (Bar Graph):** Celelalte LED-uri au fost configurate să funcționeze ca un indicator de nivel, aprinzându-se succesiv pe măsură ce valoarea numerică a comenzii crește (0-255), similar unui VU-metru audio.

5.2.2 2. Implementarea Hardware (Blocul xlconcat)

Pentru a putea transmite aceste două tipuri de informații (semnalul pulsat PWM și semnalele discrete de nivel) către un singur port fizic de ieșire (vectorul de 4 LED-uri al plăcii), a fost necesară utilizarea blocului IP `xlconcat`.

Acesta a permis agregarea semnalului PWM și a semnalelor de stare într-un singur bus de date mapat corect pe pinii fizici ai FPGA-ului, fără a necesita modificări complexe în fișierul de constrângeri.

6 Concluzii

Proiectul a avut ca obiectiv principal proiectarea și implementarea unui sistem de control automat de tip PID direct în hardware, pe o structură FPGA, utilizând aritmetica în virgulă mobilă. Această abordare a urmărit depășirea limitărilor de precizie specifice implementărilor clasice în virgulă fixă și demonstrarea capacității cipului Zynq de a gestiona calcule matematice complexe în timp real.

6.1 Contribuții Personale

Principalele realizări în cadrul acestui proiect constau în:

- **Implementarea manuală a unității de calcul (FPU):** S-au proiectat de la zero modulele VHDL pentru calcule matematice (adunare, înmulțire, împărțire), evitând utilizarea blocurilor prefabricate, pentru a asigura un control total asupra modului de procesare a datelor.
- **Sistemul de vizualizare pe LED-uri:** Deoarece testarea nu a implicat un motor real, s-a dezvoltat o soluție practică folosind LED-urile plăcii: unul indică intensitatea comenzii (prin luminozitate), iar celelalte funcționează ca un indicator de nivel pentru a vizualiza mărimea valorii.
- **Comunicarea Software-Hardware:** S-a realizat programul în limbajul C care gestionează schimbul de date dintre procesor și FPGA, asigurând sincronizarea corectă și integritatea datelor transferate.

6.2 Avantaje și Dezavantaje

Avantaje:

- **Precizie Ridicată:** Utilizarea virgulei mobile permite gestionarea unei dinamici foarte mari a valorilor, fiind esențială pentru algoritmi sensibili.
- **Determinism:** Execuția în hardware garantează un timp de răspuns constant și o stabilitate perfectă a semnalului generat (fără jitter).

Dezavantaje:

- **Consum de Resurse:** Aritmetica în virgulă mobilă ocupă o arie semnificativă pe cipul FPGA (număr mare de LUT-uri și DSP-uri).
- **Complexitatea Acordării:** Calibrarea parametrilor PID (K_p, K_i, K_d) este un proces dificil care necesită teste extinse pe sistemul fizic real.

6.3 Aplicații și Dezvoltări Viitoare

Sistemul proiectat poate servi ca bază pentru aplicații care necesită control rapid și precis, cum ar fi stabilizarea dronelor sau controlul brațelor robotice industriale.

Pentru viitor, se propun următoarele direcții de dezvoltare:

1. **Închiderea Buclei Fizice:** Conectarea unui servomotor real și a unui senzor de poziție extern.

2. **Auto-Tuning:** Implementarea unui algoritm software care să determine automat coeficienții optimi PID.

3. **Optimizarea Hardware:** Transformarea arhitecturii FPU din una secvențială într-una de tip "Pipeline" pentru frecvențe mai mari.

Bibliografie

[1] Digilent Inc., *Zybo Z7 Reference Manual*, 2023.

[2] L. H. Crockett et al., *The Zynq Book: Embedded Processing with the Arm Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC*, Strathclyde Academic Media, 2014.

[3] Xilinx Inc., *Zynq-7000 SoC Technical Reference Manual (UG585)*, 2019.

[4] K. Ogata, *Modern Control Engineering*, 5th Edition, Prentice Hall, 2010.

[5] Volnei A. Pedroni, *Circuit Design with VHDL*, MIT Press, 2004.

[6] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*, IEEE, 2019.

[7] Xilinx Inc., *7 Series FPGAs and Zynq-7000 SoC XADC Dual 12-Bit 1 MSPS Analog-to-Digital Converter User Guide (UG480)*, 2018.

[8] Katsuhiko Ogata, *Discrete-Time Control Systems*, 2nd Edition, Prentice Hall, 1995.

Anexe

Anexa A - Codul Sursă VHDL

Această anexă conține sursele VHDL implementate pentru logica programabilă (PL), organizate pe module funcționale. Comentariile au fost eliminate pentru concizie.

A.1. Nucleul Algoritmului de Control (pid_core.vhd)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity pid_core is
    generic (
        KP_VALUE : STD_LOGIC_VECTOR(31 downto 0);
        KI_VALUE : STD_LOGIC_VECTOR(31 downto 0);
        KD_VALUE : STD_LOGIC_VECTOR(31 downto 0);
        TS_VALUE : STD_LOGIC_VECTOR(31 downto 0)
    );
    port (
        clk              : in  STD_LOGIC;
        reset            : in  STD_LOGIC;
        start            : in  STD_LOGIC;
        setpoint_sp      : in  STD_LOGIC_VECTOR(31 downto 0);
        process_pv       : in  STD_LOGIC_VECTOR(31 downto 0);
        control_output_u : out STD_LOGIC_VECTOR(31 downto 0);
        done             : out STD_LOGIC
    );
end pid_core;

architecture Behavioral of pid_core is

    component vm_add_sub is
    port(
        clk      : in  std_logic;
        reset    : in  std_logic;
        start    : in  std_logic;
        subtract : in  std_logic;
        A        : in  std_logic_vector(31 downto 0);
        B        : in  std_logic_vector(31 downto 0);
        done     : out std_logic;
        result   : out std_logic_vector(31 downto 0)
    );
    end component;

    component vm_mul is
    port (
        clk   : in  STD_LOGIC;
        start : in  STD_LOGIC;
        A     : in  STD_LOGIC_VECTOR(31 downto 0);
```

```
        B     : in  STD_LOGIC_VECTOR(31 downto 0);
        Z     : out STD_LOGIC_VECTOR(31 downto 0);
        done  : out STD_LOGIC
    );
    end component;

    component vm_div is
    port (
        clk   : in  STD_LOGIC;
        start : in  STD_LOGIC;
        A     : in  STD_LOGIC_VECTOR(31 downto 0);
        B     : in  STD_LOGIC_VECTOR(31 downto 0);
        Z     : out STD_LOGIC_VECTOR(31 downto 0);
        done  : out STD_LOGIC
    );
    end component;

    type STATE_TYPE is (
        IDLE,
        CALC_ERROR,
        CALC_P,
        CALC_I_PART1,
        CALC_I_PART2,
        CALC_I_ADD,
        CALC_D_DIFF,
        CALC_D_MUL,
        CALC_D_DIV,
        CALC_U_ADD1,
        CALC_U_ADD2,
        UPDATE_ERROR,
        FINISHED
    );
    signal state : STATE_TYPE := IDLE;

    signal error_n       : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
    signal error_n_1     : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
    signal integral_sum : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');

    signal fp_A, fp_B     : STD_LOGIC_VECTOR(31 downto 0);
    signal fp_sub         : STD_LOGIC := '0';

    signal fp_res_add     : STD_LOGIC_VECTOR(31 downto 0);
    signal fp_res_mul     : STD_LOGIC_VECTOR(31 downto 0);
    signal fp_res_div     : STD_LOGIC_VECTOR(31 downto 0);

    signal start_add, start_mul, start_div : STD_LOGIC := '0';
    signal done_add, done_mul, done_div    : STD_LOGIC := '0';

```

```
    signal term_P, term_I, term_D : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
    signal temp_val_i             : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
    signal temp_val_d             : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');

begin

    ADD_UNIT : vm_add_sub
        port map (
            clk => clk, reset => reset, start => start_add, subtract => fp_sub,
            A => fp_A, B => fp_B, done => done_add, result => fp_res_add
        );

    MUL_UNIT : vm_mul
        port map (
            clk => clk, start => start_mul,
            A => fp_A, B => fp_B, Z => fp_res_mul, done => done_mul
        );

    DIV_UNIT : vm_div
        port map (
            clk => clk, start => start_div,
            A => fp_A, B => fp_B, Z => fp_res_div, done => done_div
        );

    process(clk, reset)
    begin
        if reset = '1' then
            state <= IDLE;
            integral_sum <= (others => '0');
            error_n_1    <= (others => '0');
            done         <= '0';
            control_output_u <= (others => '0');
        elsif rising_edge(clk) then
            start_add <= '0';
            start_mul <= '0';
            start_div <= '0';
            done      <= '0';

            case state is
                when IDLE =>
                    if start = '1' then
                        state <= CALC_ERROR;
                    end if;

```

```
                when CALC_ERROR =>
                    fp_A <= setpoint_sp;
                    fp_B <= process_pv;
                    fp_sub <= '1';
                    start_add <= '1';

                    if done_add = '1' then
                        error_n <= fp_res_add;
                        state <= CALC_P;
                    end if;

                when CALC_P =>
                    fp_A <= KP_VALUE;
                    fp_B <= error_n;
                    start_mul <= '1';

                    if done_mul = '1' then
                        term_P <= fp_res_mul;
                        state <= CALC_I_PART1;
                    end if;

                when CALC_I_PART1 =>
                    fp_A <= KI_VALUE;
                    fp_B <= error_n;
                    start_mul <= '1';

                    if done_mul = '1' then
                        temp_val_i <= fp_res_mul;
                        state <= CALC_I_PART2;
                    end if;

                when CALC_I_PART2 =>
                    fp_A <= temp_val_i;
                    fp_B <= TS_VALUE;
                    start_mul <= '1';

                    if done_mul = '1' then
                        temp_val_i <= fp_res_mul;
                        state <= CALC_I_ADD;
                    end if;

                when CALC_I_ADD =>
                    fp_A <= integral_sum;
                    fp_B <= temp_val_i;
                    fp_sub <= '0';
                    start_add <= '1';

                    if done_add = '1' then
                        integral_sum <= fp_res_add;
                        term_I <= fp_res_add;
                        state <= CALC_D_DIFF;
```

```
                    end if;

                when CALC_D_DIFF =>
                    fp_A <= error_n;
                    fp_B <= error_n_1;
                    fp_sub <= '1';
                    start_add <= '1';

                    if done_add = '1' then
                        temp_val_d <= fp_res_add;
                        state <= CALC_D_MUL;
                    end if;

                when CALC_D_MUL =>
                    fp_A <= KD_VALUE;
                    fp_B <= temp_val_d;
                    start_mul <= '1';

                    if done_mul = '1' then
                        temp_val_d <= fp_res_mul;
                        state <= CALC_D_DIV;
                    end if;

                when CALC_D_DIV =>
                    fp_A <= temp_val_d;
                    fp_B <= TS_VALUE;
                    start_div <= '1';

                    if done_div = '1' then
                        term_D <= fp_res_div;
                        state <= CALC_U_ADD1;
                    end if;

                when CALC_U_ADD1 =>
                    fp_A <= term_P;
                    fp_B <= term_I;
                    fp_sub <= '0';
                    start_add <= '1';

                    if done_add = '1' then
                        temp_val_i <= fp_res_add;
                        state <= CALC_U_ADD2;
                    end if;

                when CALC_U_ADD2 =>
                    fp_A <= temp_val_i;
                    fp_B <= term_D;
                    fp_sub <= '0';
                    start_add <= '1';

                    if done_add = '1' then
```

```
                        control_output_u <= fp_res_add;
                        state <= UPDATE_ERROR;
                    end if;

                when UPDATE_ERROR =>
                    error_n_1 <= error_n;
                    state <= FINISHED;

                when FINISHED =>
                    done <= '1';
                    if start = '0' then
                        state <= IDLE;
                    end if;

            end case;
        end if;
    end process;
end Behavioral;
```

A.2. Unități Aritmetice în Virgulă Mobilă

Sumator/Scăzător (vm_add_sub.vhd)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity vm_add_sub is
    port(
        A        : in  std_logic_vector(31 downto 0);
        B        : in  std_logic_vector(31 downto 0);
        subtract : in  std_logic;
        clk      : in  std_logic;
        reset    : in  std_logic;
        start    : in  std_logic;
        done     : out std_logic;
        result   : out std_logic_vector(31 downto 0)
    );
end vm_add_sub;

architecture Behavioral of vm_add_sub is

    type ST is (WAIT_STATE, UNPACK_STATE, ALIGN_STATE,
       ADDITION_STATE, NORMALIZE_STATE, PACK_STATE);
    signal state : ST := WAIT_STATE;

    constant MANT_WIDTH : integer := 28;
    signal A_mantissa_ext, B_mantissa_ext : unsigned(MANT_WIDTH-1
        downto 0);

    signal A_exp_s, B_exp_s : signed(8 downto 0);
```

```
    signal A_sgn, B_sgn     : std_logic;

    signal final_exp_s    : signed(8 downto 0);
    signal final_mantissa : signed(MANT_WIDTH downto 0);
    signal final_sgn      : std_logic;

    constant EXP_BIAS : signed(8 downto 0) := to_signed(127, 9);

    constant INF_MANT : STD_LOGIC_VECTOR(31 downto 0) := X"7F800000";
    constant ZERO_MANT: STD_LOGIC_VECTOR(31 downto 0) := X"00000000";

begin

    Control_Unit : process(clk, reset)
        variable diff : signed(8 downto 0);
        variable shift_amount : integer range 0 to MANT_WIDTH;
        variable temp_A_mantissa, temp_B_mantissa : unsigned(MANT_WIDTH-1 downto 0);
        variable temp_mantissa_result : signed(MANT_WIDTH downto 0);
        variable temp_final_sgn_v     : std_logic;

    begin
        if (reset = '1') then
            state <= WAIT_STATE;
            done  <= '0';
        elsif rising_edge(clk) then

            temp_A_mantissa := A_mantissa_ext;
            temp_B_mantissa := B_mantissa_ext;

            case state is

                when WAIT_STATE =>
                    done <= '0';
                    if (start = '1') then state <= UNPACK_STATE; end if;

                when UNPACK_STATE =>
                    A_sgn      <= A(31);
                    B_sgn      <= B(31) xor subtract;
                    A_exp_s    <= to_signed(to_integer(unsigned('0' & A(30 downto 23))), 9);
                    B_exp_s    <= to_signed(to_integer(unsigned('0' & B(30 downto 23))), 9);

                    A_mantissa_ext <= '1' & unsigned(A(22 downto 0)) & "0000";
                    B_mantissa_ext <= '1' & unsigned(B(22 downto
```

```
                0)) & "0000";

            state <= ALIGN_STATE;

        when ALIGN_STATE =>
            if A_exp_s > B_exp_s then
                diff := A_exp_s - B_exp_s;
                final_exp_s <= A_exp_s;
                shift_amount := to_integer(diff);
                if shift_amount >= MANT_WIDTH then
                    temp_final_sgn_v := A_sgn;
                    final_mantissa <= signed('0' &
                       A_mantissa_ext);
                    state <= PACK_STATE;
                else
                    temp_B_mantissa := shift_right(
                       B_mantissa_ext, shift_amount);
                    state <= ADDITION_STATE;
                end if;
            elsif A_exp_s < B_exp_s then
                diff := B_exp_s - A_exp_s;
                final_exp_s <= B_exp_s;
                shift_amount := to_integer(diff);
                if shift_amount >= MANT_WIDTH then
                    temp_final_sgn_v := B_sgn;
                    final_mantissa <= signed('0' &
                       B_mantissa_ext);
                    state <= PACK_STATE;
                else
                    temp_A_mantissa := shift_right(
                       A_mantissa_ext, shift_amount);
                    state <= ADDITION_STATE;
                end if;
            else
                final_exp_s <= A_exp_s;
                state <= ADDITION_STATE;
            end if;

            A_mantissa_ext <= temp_A_mantissa;
            B_mantissa_ext <= temp_B_mantissa;
            if state = PACK_STATE then final_sgn <=
               temp_final_sgn_v; end if;

        when ADDITION_STATE =>

            if A_sgn = B_sgn then
                temp_mantissa_result := signed('0' &
                   A_mantissa_ext) + signed('0' &
                   B_mantissa_ext);
                temp_final_sgn_v     := A_sgn;
            elsif A_mantissa_ext >= B_mantissa_ext then
```

```
                        temp_mantissa_result := signed('0' & A_mantissa_ext) - signed('0' & B_mantissa_ext);
                        temp_final_sgn_v     := A_sgn;
                    else
                        temp_mantissa_result := signed('0' & B_mantissa_ext) - signed('0' & A_mantissa_ext);
                        temp_final_sgn_v     := B_sgn;
                    end if;

                    final_mantissa <= temp_mantissa_result;
                    final_sgn      <= temp_final_sgn_v;
                    state <= NORMALIZE_STATE;

                when NORMALIZE_STATE =>
                    if to_unsigned(0, MANT_WIDTH+1) = unsigned(final_mantissa) then
                        final_exp_s <= (others => '0');
                        state <= PACK_STATE;

                    elsif final_mantissa(MANT_WIDTH) = '1' then
                        final_mantissa <= shift_right(final_mantissa, 1);
                        final_exp_s    <= final_exp_s + 1;
                        state <= PACK_STATE;

                    elsif final_mantissa(MANT_WIDTH-1) = '0' then
                        for i in MANT_WIDTH-2 downto 0 loop
                            if final_mantissa(i) = '1' then
                                shift_amount := (MANT_WIDTH-1) - i;
                                exit;
                            end if;
                        end loop;

                        if shift_amount /= 0 then
                            final_mantissa <= shift_left(final_mantissa, shift_amount);
                            final_exp_s    <= final_exp_s - to_signed(shift_amount, 9);
                        end if;
                        state <= PACK_STATE;
                    else
                        state <= PACK_STATE;
                    end if;

                when PACK_STATE =>
                    if final_exp_s > to_signed(255, 9) then
                        result <= final_sgn & INF_MANT(30 downto 0);
```

```
                    elsif final_exp_s <= to_signed(0, 9) then
                        result <= final_sgn & ZERO_MANT(30 downto 0);
                    else
                        result(31)           <= final_sgn;
                        result(30 downto 23) <= std_logic_vector(final_exp_s(7 downto 0));
                        result(22 downto 0)  <= std_logic_vector(final_mantissa(26 downto 4));
                    end if;

                    done  <= '1';
                    if (start = '0') then state <= WAIT_STATE; end if;

            end case;
        end if;
    end process;
end Behavioral;
```

Înmulțitor (vm_mul.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity vm_mul is
    Port (
        clk     : in  STD_LOGIC;
        start   : in  STD_LOGIC;
        A       : in  STD_LOGIC_VECTOR(31 downto 0);
        B       : in  STD_LOGIC_VECTOR(31 downto 0);
        Z       : out STD_LOGIC_VECTOR(31 downto 0);
        done    : out STD_LOGIC
    );
end vm_mul;

architecture Behavioral of vm_mul is

    type state_type is (IDLE, UNPACK, CHECK_SPECIAL, MULTIPLY_MANT, NORMALIZE, PACK, FINISHED);
    signal state : state_type := IDLE;

    signal signA, signB, signZ : STD_LOGIC;
    signal expA, expB : unsigned(7 downto 0);
    signal expResult : signed(9 downto 0);
    signal mantA, mantB : unsigned(23 downto 0);
    signal mantProd : unsigned(47 downto 0);
    signal final_mant : unsigned(22 downto 0);
    signal final_exp  : signed(9 downto 0);
    signal result : STD_LOGIC_VECTOR(31 downto 0);

```

```
    constant INF_VAL : STD_LOGIC_VECTOR(31 downto 0) := X"7F800000";
    constant NAN_VAL : STD_LOGIC_VECTOR(31 downto 0) := X"7FC00000";
    constant ZERO_VAL: STD_LOGIC_VECTOR(31 downto 0) := X"00000000";

begin

    Z <= result;

    process(clk)
        variable v_temp_mant : unsigned(23 downto 0);
        variable v_guard     : std_logic;
    begin
        if rising_edge(clk) then
            case state is
                when IDLE =>
                    done <= '0';
                    if start = '1' then
                        state <= UNPACK;
                    end if;

                when UNPACK =>
                    signA <= A(31);
                    signB <= B(31);
                    expA  <= unsigned(A(30 downto 23));
                    expB  <= unsigned(B(30 downto 23));

                    if unsigned(A(30 downto 23)) /= 0 then
                        mantA <= '1' & unsigned(A(22 downto 0));
                    else
                        mantA <= '0' & unsigned(A(22 downto 0));
                    end if;

                    if unsigned(B(30 downto 23)) /= 0 then
                        mantB <= '1' & unsigned(B(22 downto 0));
                    else
                        mantB <= '0' & unsigned(B(22 downto 0));
                    end if;

                    state <= CHECK_SPECIAL;

                when CHECK_SPECIAL =>
                    signZ <= signA xor signB;

                    if (expA = 255) or (expB = 255) then
                        result <= INF_VAL;
                        state <= FINISHED;
                    elsif (expA = 0) or (expB = 0) then
                        result <= ZERO_VAL;
```

```
                        state <= FINISHED;
                    else
                        state <= MULTIPLY_MANT;
                    end if;

                when MULTIPLY_MANT =>
                    mantProd <= mantA * mantB;
                    expResult <= signed('0' & '0' & std_logic_vector(expA)) +
                                 signed('0' & '0' & std_logic_vector(expB)) - 127;
                    state <= NORMALIZE;

                when NORMALIZE =>
                    if mantProd(47) = '1' then
                        v_temp_mant := mantProd(47 downto 24);
                        v_guard     := mantProd(23);
                        final_exp   <= expResult + 1;
                    else
                        v_temp_mant := mantProd(46 downto 23);
                        v_guard     := mantProd(22);
                        final_exp   <= expResult;
                    end if;

                    if v_guard = '1' then
                        v_temp_mant := v_temp_mant + 1;
                    end if;

                    final_mant <= v_temp_mant(22 downto 0);
                    state <= PACK;

                when PACK =>
                    result(31) <= signZ;
                    result(30 downto 23) <= std_logic_vector(final_exp(7 downto 0));
                    result(22 downto 0)  <= std_logic_vector(final_mant);
                    state <= FINISHED;

                when FINISHED =>
                    done <= '1';
                    if start = '0' then
                        state <= IDLE;
                    end if;
            end case;
        end if;
    end process;
end Behavioral;
```

Divizor (vm_div.vhd)

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity vm_div is
    Port (
        clk     : in  STD_LOGIC;
        start   : in  STD_LOGIC;
        A       : in  STD_LOGIC_VECTOR(31 downto 0);
        B       : in  STD_LOGIC_VECTOR(31 downto 0);
        Z       : out STD_LOGIC_VECTOR(31 downto 0);
        done    : out STD_LOGIC
    );
end vm_div;

architecture Behavioral of vm_div is

    type state_type is (IDLE, UNPACK, CHECK_SPECIAL, DIVIDE_MANT,
        SUB_EXP, NORMALIZE, ROUND_STAGE, PACK, FINISHED);
    signal state : state_type := IDLE;

    signal signA, signB, signZ : STD_LOGIC;
    signal expA, expB : unsigned(7 downto 0);
    signal expDiff    : signed(9 downto 0);
    signal mantA, mantB : unsigned(23 downto 0);
    signal mantA_ext : unsigned(47 downto 0);
    signal mantQuot  : unsigned(47 downto 0);
    signal mantNorm_temp : unsigned(23 downto 0);
    signal mantNorm      : unsigned(22 downto 0);
    signal result        : STD_LOGIC_VECTOR(31 downto 0);

    constant EXP_BIAS : integer := 127;
    constant ALL_ZEROS_23 : STD_LOGIC_VECTOR(22 downto 0) := (
       others => '0');
    constant INF_VAL_31   : STD_LOGIC_VECTOR(30 downto 0) := "
       11111111" & "00000000000000000000000";

begin

    Z <= result;

    process(clk)
    begin
        if rising_edge(clk) then
            case state is
                when IDLE =>
                    done <= '0';
                    if start = '1' then state <= UNPACK; end if;

                when UNPACK =>
                    signA <= A(31); signB <= B(31);
                    expA <= unsigned(A(30 downto 23)); expB <=
```

```
                unsigned(B(30 downto 23));

            if unsigned(A(30 downto 23)) /= 0 then mantA <= '1' & unsigned(A(22 downto 0));
            else mantA <= '0' & unsigned(A(22 downto 0)); end if;

            if unsigned(B(30 downto 23)) /= 0 then mantB <= '1' & unsigned(B(22 downto 0));
            else mantB <= '0' & unsigned(B(22 downto 0)); end if;
            state <= CHECK_SPECIAL;

        when CHECK_SPECIAL =>
            signZ <= signA xor signB;

            if (expA = 255 and A(22 downto 0) /= ALL_ZEROS_23) or
               (expB = 255 and B(22 downto 0) /= ALL_ZEROS_23) then
                result <= X"7FC00000"; state <= FINISHED;

            elsif (expA=0 and A(22 downto 0)=ALL_ZEROS_23 and expB=0 and B(22 downto 0)=ALL_ZEROS_23) or
                (expA=255 and expB=255) then
                result <= X"7FC00000"; state <= FINISHED;

            elsif (expB=0 and B(22 downto 0)=ALL_ZEROS_23) or (expA=255) then
                result <= (signA xor signB) & INF_VAL_31; state <= FINISHED;

            elsif (expA=0 and A(22 downto 0)=ALL_ZEROS_23) then
                result(31) <= signA xor signB; result(30 downto 0) <= (others=>'0'); state <= FINISHED;
            else
                state <= DIVIDE_MANT;
            end if;

        when DIVIDE_MANT =>
            mantA_ext <= mantA & to_unsigned(0, 24);
            mantQuot <= (mantA & to_unsigned(0, 24)) / mantB;
            state <= SUB_EXP;

        when SUB_EXP =>
            expDiff <= signed('0' & '0' & std_logic_vector(expA)) -
```

```
                                signed('0' & '0' &
                                   std_logic_vector(expB)) +
                                to_signed(EXP_BIAS, 10);
                        state <= NORMALIZE;

                    when NORMALIZE =>
                        if mantQuot(24) = '1' then
                            mantNorm_temp <= mantQuot(24 downto 1);
                        else
                            mantNorm_temp <= mantQuot(23 downto 0);
                            expDiff <= expDiff - 1;
                        end if;
                        state <= ROUND_STAGE;

                    when ROUND_STAGE =>
                        mantNorm <= mantNorm_temp(22 downto 0);
                        state <= PACK;

                    when PACK =>
                        if expDiff >= to_signed(255, 10) then
                            result <= signZ & INF_VAL_31;
                        elsif expDiff <= to_signed(0, 10) then
                            result(31) <= signZ;
                            result(30 downto 0) <= (others => '0');
                        else
                            result <= signZ & std_logic_vector(
                               expDiff(7 downto 0)) & std_logic_vector
                               (mantNorm);
                        end if;
                        state <= FINISHED;

                    when FINISHED =>
                        done <= '1';
                        if start = '0' then state <= IDLE; end if;

                end case;
            end if;
        end process;
end Behavioral;
```

A.3. Convertor Date (fp_converter.vhd)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity fp_converter is
    port (
        clk       : in  std_logic;
        reset     : in  std_logic;

```

```
        start_i2f : in  std_logic;
        in_int    : in  std_logic_vector(11 downto 0);
        out_fp    : out std_logic_vector(31 downto 0);
        done_i2f  : out std_logic;

        start_f2i : in  std_logic;
        in_fp     : in  std_logic_vector(31 downto 0);
        out_int   : out std_logic_vector(11 downto 0);
        done_f2i  : out std_logic
    );
end fp_converter;

architecture Behavioral of fp_converter is

    type CONV_STATE is (C_IDLE, C_I2F_FIND_MSB, C_I2F_PACK,
       C_F2I_UNPACK, C_F2I_FINISH);
    signal state : CONV_STATE := C_IDLE;

    signal working_int : unsigned(31 downto 0) := (others => '0')
       ;

    signal fp_sign : std_logic := '0';
    signal fp_exp_u : unsigned(7 downto 0) := (others => '0');
    signal fp_mant_u : unsigned(22 downto 0) := (others => '0');

    constant EXP_BIAS : integer := 127;
    constant MAX_12_BIT : unsigned(11 downto 0) := (others =>
       '1');

begin

    process(clk, reset)
        variable shift_count_v : integer range 0 to 31;
        variable temp_fp_exp_v : integer;
        variable temp_shifted_v : unsigned(31 downto 0);
        variable temp_int_v : unsigned(31 downto 0);
        variable clamped_value_v : unsigned(11 downto 0);
    begin
        if reset = '1' then
            state <= C_IDLE;
            done_i2f <= '0';
            done_f2i <= '0';
            out_fp <= (others => '0');
            out_int <= (others => '0');
            working_int <= (others => '0');
        elsif rising_edge(clk) then
            done_i2f <= '0';
            done_f2i <= '0';

            case state is
                when C_IDLE =>
```

```
                    if start_i2f = '1' then
                        fp_sign <= '0';
                        working_int <= resize(unsigned(in_int), 32);
                        state <= C_I2F_FIND_MSB;
                    elsif start_f2i = '1' then
                        fp_sign <= in_fp(31);
                        state <= C_F2I_UNPACK;
                    end if;

                when C_I2F_FIND_MSB =>
                    shift_count_v := 0;
                    for i in 30 downto 0 loop
                        if working_int(i) = '1' then
                            shift_count_v := i;
                            exit;
                        end if;
                    end loop;

                    if shift_count_v = 0 and working_int = 0 then
                        fp_exp_u <= (others => '0');
                        fp_mant_u <= (others => '0');
                    else
                        temp_fp_exp_v := EXP_BIAS + shift_count_v;
                        fp_exp_u <= to_unsigned(temp_fp_exp_v, 8);

                        if shift_count_v < 23 then
                            temp_shifted_v := shift_left(working_int, 23 - shift_count_v);
                            fp_mant_u <= temp_shifted_v(22 downto 0);
                        else
                            fp_mant_u <= (others => '0');
                        end if;
                    end if;
                    state <= C_I2F_PACK;

                when C_I2F_PACK =>
                    out_fp(31) <= fp_sign;
                    out_fp(30 downto 23) <= std_logic_vector(fp_exp_u);
                    out_fp(22 downto 0) <= std_logic_vector(fp_mant_u);
                    done_i2f <= '1';
                    state <= C_IDLE;

                when C_F2I_UNPACK =>
                    temp_fp_exp_v := to_integer(unsigned(in_fp(30 downto 23))) - EXP_BIAS;
```

```
                    if fp_sign = '1' or temp_fp_exp_v < 0 then
                        working_int <= (others => '0');

                    elsif temp_fp_exp_v > 11 then
                        working_int <= (others => '1');

                    else
                        temp_int_v := (others => '0');
                        temp_int_v(23 downto 0) := unsigned('1' &
                            in_fp(22 downto 0));
                        temp_int_v := shift_right(temp_int_v, 23
                            - temp_fp_exp_v);
                        working_int <= temp_int_v;
                    end if;

                    state <= C_F2I_FINISH;

                when C_F2I_FINISH =>
                    if working_int > 4095 then
                        clamped_value_v := MAX_12_BIT;
                    else
                        clamped_value_v := working_int(11 downto
                            0);
                    end if;

                    out_int <= std_logic_vector(clamped_value_v);
                    done_f2i <= '1';
                    state <= C_IDLE;

                when others =>
                    state <= C_IDLE;
            end case;
        end if;
    end process;
end Behavioral;
```

A.4. Sistemul de Scalare PWM (conv_pwm_int.vhd)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity conv_pwm_int is
    port (
        clk      : in  std_logic;
        reset    : in  std_logic;
        start    : in  std_logic;
        in_val   : in  std_logic_vector(11 downto 0);
        pwm_val  : out std_logic_vector(7 downto 0);
        gata     : out std_logic
```

```
    );
end entity conv_pwm_int;

architecture Behavioral of conv_pwm_int is

    signal mult_out   : std_logic_vector(24 downto 0);
    signal mult_gata  : std_logic;
    signal div_out    : std_logic_vector(11 downto 0);
    signal div_gata   : std_logic;

    component inmultitor_shift_add is
        port (
            clk      : in std_logic;
            resetare : in std_logic;
            start    : in std_logic;
            a        : in std_logic_vector(11 downto 0);
            b        : in std_logic_vector(11 downto 0);
            rezultat : out std_logic_vector(24 downto 0);
            gata     : out std_logic
        );
    end component;

    component RestoringDivision is
        port (
            clk       : in std_logic;
            resetare  : in std_logic;
            start     : in std_logic;
            dividend  : in std_logic_vector(24 downto 0);
            divisor   : in std_logic_vector(11 downto 0);
            quotient  : out std_logic_vector(11 downto 0);
            gata      : out std_logic
        );
    end component;

begin

    mult_inst : inmultitor_shift_add
        port map (
            clk      => clk,
            resetare => reset,
            start    => start,
            a        => in_val,
            b        => "000011111111",
            rezultat => mult_out,
            gata     => mult_gata
        );

    div_inst : RestoringDivision
        port map (
            clk       => clk,
            resetare  => reset,
```

```
            start     => mult_gata,
            dividend  => mult_out,
            divisor   => "111111111111",
            quotient  => div_out,
            gata      => div_gata
        );

    pwm_val <= div_out(7 downto 0);
    gata    <= div_gata;

end architecture Behavioral;
```

Sub-modul Înmulțitor (inmultitor_shift_add.vhd)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity inmultitor_shift_add is
    generic (
        N : integer := 12
    );
    port (
        clk      : in  std_logic;
        resetare : in  std_logic;
        start    : in  std_logic;
        a        : in  std_logic_vector(N-1 downto 0);
        b        : in  std_logic_vector(N-1 downto 0);
        rezultat: out std_logic_vector(2*N downto 0);
        gata     : out std_logic
    );
end entity inmultitor_shift_add;

architecture Behavioral of inmultitor_shift_add is
    type stare_t is (ASTEPTARE, CALCUL, FINALIZARE);
    signal stare : stare_t;

    signal produs : unsigned(2*N downto 0);
    signal multiplicand : unsigned(2*N downto 0);
    signal multiplicator : unsigned(N-1 downto 0);
    signal contor : integer range 0 to N;

begin
    process(clk, resetare)
    begin
        if resetare = '1' then
            stare <= ASTEPTARE;
            produs <= (others => '0');
            multiplicand <= (others => '0');
            multiplicator <= (others => '0');
            contor <= 0;
            gata <= '0';
```

```
        elsif rising_edge(clk) then
            case stare is
                when ASTEPTARE =>
                    if start = '1' then
                        stare <= CALCUL;
                        produs <= (others => '0');
                        multiplicand <= resize(unsigned(a), 2*N +
                            1);
                        multiplicator <= unsigned(b);
                        contor <= 0;
                        gata <= '0';
                    end if;

                when CALCUL =>
                    if multiplicator(0) = '1' then
                        produs <= produs + multiplicand;
                    end if;

                    multiplicand <= shift_left(multiplicand, 1);
                    multiplicator <= shift_right(multiplicator,
                       1);

                    contor <= contor + 1;
                    if contor = N-1 then
                        stare <= FINALIZARE;
                    end if;

                when FINALIZARE =>
                    rezultat <= std_logic_vector(produs);
                    gata <= '1';
                    stare <= ASTEPTARE;
            end case;
        end if;
    end process;
end architecture Behavioral;
```

Sub-modul Divizor (RestoringDivision.vhd)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.std_logic_unsigned.all;

entity RestoringDivision is
    generic (
        N_QUOTIENT : integer := 12;
        N_DIVIDEND : integer := 25
    );
    port (
        clk      : in  std_logic;
        resetare : in  std_logic;
        start    : in  std_logic;
```

```
        dividend : in  std_logic_vector(N_DIVIDEND-1 downto 0);
        divisor  : in  std_logic_vector(N_QUOTIENT-1 downto 0);
        quotient : out std_logic_vector(N_QUOTIENT-1 downto 0);
        gata     : out std_logic
    );
end entity RestoringDivision;

architecture Behavioral of RestoringDivision is

    constant N_AM : integer := N_QUOTIENT;
    constant N_AM_EXT : integer := N_QUOTIENT + 1;

    type state_type is (S_ASTEPTARE, S_CALCUL, S_FINAL);
    signal state  : state_type;
    signal A      : std_logic_vector(N_AM_EXT-1 downto 0);
    signal M      : std_logic_vector(N_AM_EXT-1 downto 0);
    signal Q      : std_logic_vector(N_QUOTIENT-1 downto 0);
    signal R : std_logic_vector(N_DIVIDEND-1 downto 0);
    signal contor : integer range 0 to N_QUOTIENT;

begin
    process(clk, resetare)
        variable A_prim, A_partial : std_logic_vector(N_AM_EXT-1
           downto 0);
        variable R_partial : std_logic_vector(N_DIVIDEND-1 downto
            0);
        variable currentCont : integer range 0 to N_QUOTIENT;
    begin
        if resetare = '1' then
            state       <= S_ASTEPTARE;
            M           <= (others => '0');
            Q           <= (others => '0');
            R           <= (others => '0');
            A           <= (others => '0');
            gata        <= '0';
            contor      <= 0;
        elsif rising_edge(clk) then
            case state is
                when S_ASTEPTARE =>
                    if start = '1' then
                        R <= dividend;
                        M <= (others => '0');
                        M(N_QUOTIENT-1 downto 0) <= divisor;
                        M <= (not M) + 1;
                        gata <= '0';
                        Q <= (others => '0');
                        A <= (others => '0');
                        contor <= N_QUOTIENT;
                        state  <= S_CALCUL;
                    end if;

```

```
                when S_CALCUL =>
                    currentCont := contor;
                    A_partial := A;
                    R_partial := R;

                    A_prim := A_partial(N_AM_EXT-2 downto 0) &
                       R_partial(N_DIVIDEND-1);
                    R_partial(N_DIVIDEND-1 downto 1) := R_partial
                       (N_DIVIDEND-2 downto 0);
                    R_partial(0) := '0';

                    A_partial := A_prim;
                    A_prim := A_prim + M;

                    if A_prim(N_AM_EXT-1) = '1' then
                        A_prim := A_partial;
                        R_partial(0) := '0';
                    else
                        R_partial(0) := '1';
                    end if;

                    A <= A_prim;
                    R <= R_partial;

                    currentCont := currentCont - 1;
                    contor <= currentCont;

                    if currentCont = 0 then
                        state <= S_FINAL;
                    end if;

                when S_FINAL =>
                    quotient <= R(N_QUOTIENT-1 downto 0);
                    gata     <= '1';
                    state    <= S_ASTEPTARE;

            end case;
        end if;
    end process;
end architecture Behavioral;
```

A.5. Generatorul PWM (servo_pwm_gen.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity servo_pwm_gen is
    Port (
        clk       : in  STD_LOGIC;
        val_in    : in  STD_LOGIC_VECTOR (7 downto 0);
```

```
        servo_pin : out STD_LOGIC
    );
end servo_pwm_gen;

architecture Behavioral of servo_pwm_gen is
    constant PWM_PERIOD_TICKS : integer := 2500000;
    constant POS_MIN_TICKS    : integer := 125000;
    constant TICKS_PER_STEP   : integer := 490;

    signal cnt       : integer range 0 to PWM_PERIOD_TICKS := 0;
    signal high_time : integer range 0 to 250000 := 0;

begin
    high_time <= POS_MIN_TICKS + (to_integer(unsigned(val_in)) *
       TICKS_PER_STEP);

    process(clk)
    begin
        if rising_edge(clk) then
            if cnt < PWM_PERIOD_TICKS - 1 then
                cnt <= cnt + 1;
            else
                cnt <= 0;
            end if;

            if cnt < high_time then
                servo_pin <= '1';
            else
                servo_pin <= '0';
            end if;
        end if;
    end process;
end Behavioral;
```

Anexa B - Fișierul de Constrângeri (.xdc)

Această anexă conține maparea porturilor logice pe pinii fizici ai plăcii Zybo Z7. Au fost păstrate doar liniile active utilizate în proiect.

```
##LEDs
set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [
   get_ports { leds_4bits[0] }];
set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [
   get_ports { leds_4bits[1] }];
set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [
   get_ports { leds_4bits[2] }];
set_property -dict { PACKAGE_PIN D18   IOSTANDARD LVCMOS33 } [
   get_ports { leds_4bits[3] }];

##Pmod Header JA (XADC)
```

```
set_property -dict { PACKAGE_PIN N15   IOSTANDARD LVCMOS33 } [
   get_ports { vaux14p }];
set_property -dict { PACKAGE_PIN L14   IOSTANDARD LVCMOS33 } [
   get_ports { vaux7p }];
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [
   get_ports { vaux15p }];
set_property -dict { PACKAGE_PIN K14   IOSTANDARD LVCMOS33 } [
   get_ports { vaux6p }];
set_property -dict { PACKAGE_PIN N16   IOSTANDARD LVCMOS33 } [
   get_ports { vaux14n }];
set_property -dict { PACKAGE_PIN L15   IOSTANDARD LVCMOS33 } [
   get_ports { vaux7n }];
set_property -dict { PACKAGE_PIN J16   IOSTANDARD LVCMOS33 } [
   get_ports { vaux15n }];
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [
   get_ports { vaux6n }];

##Pmod Header JE (Servo Output)
set_property -dict { PACKAGE_PIN V12   IOSTANDARD LVCMOS33 } [
   get_ports {servo_signal}];
```

Anexa C - Codul Sursă Software (main.c)

Programul principal rulat pe procesorul ARM Cortex-A9, responsabil de orchestrarea datelor între XADC și acceleratorul hardware.

```
#include <stdio.h>
#include <stdint.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xadcps.h"
#include "xgpio.h"
#include "sleep.h"

#define XADC_DEVICE_ID  0
#define LSB_BITS_NOT_USED   4
#define CH_VAUX14           30

XGpio gpio_rst_conv, gpio_rst_fp, gpio_start;
XGpio gpio_pot_val, gpio_pwm_val, gpio_done;

XAdcPs_Config *xadc_cfg;
static XAdcPs xadc_ctl;

void print_float_custom(char* label, float val) {
    int dec = (int)val;
    int frac = (int)((val - dec) * 1000);
    if (frac < 0) frac = -frac;
    xil_printf("%s:␣%d.%03d", label, dec, frac);
}
```

```

int main(){
    init_platform();

    xil_printf("--- START SISTEM SERVO PID (FARA LED SW) ---\n");

    int status = 0;
    status |= XGpio_Initialize(&gpio_rst_conv, XPAR_AXI_GPIO_RST_CONV_BASEADDR);
    status |= XGpio_Initialize(&gpio_rst_fp, XPAR_AXI_GPIO_RST_FP_BASEADDR);
    status |= XGpio_Initialize(&gpio_start, XPAR_AXI_GPIO_START_BASEADDR);
    status |= XGpio_Initialize(&gpio_pot_val, XPAR_AXI_GPIO_POT_VAL_BASEADDR);
    status |= XGpio_Initialize(&gpio_pwm_val, XPAR_AXI_GPIO_PWM_VAL_BASEADDR);
    status |= XGpio_Initialize(&gpio_done, XPAR_AXI_GPIO_DONE_BASEADDR);

    if(status != XST_SUCCESS){
        xil_printf("Eroare init GPIO!\n");
        return -1;
    }

    XGpio_SetDataDirection(&gpio_rst_conv, 1, 0x0);
    XGpio_SetDataDirection(&gpio_rst_fp,   1, 0x0);
    XGpio_SetDataDirection(&gpio_start,    1, 0x0);
    XGpio_SetDataDirection(&gpio_pot_val,  1, 0x0);

    XGpio_SetDataDirection(&gpio_pwm_val,  1, 0xFFFFFFFF);
    XGpio_SetDataDirection(&gpio_done,     1, 0xFFFFFFFF);

    xadc_cfg = XAdcPs_LookupConfig(XADC_DEVICE_ID);
    if(xadc_cfg == NULL) {
        xil_printf("Eroare init XADC!\n");
        return -1;
    }

    XAdcPs_CfgInitialize(&xadc_ctl, xadc_cfg, xadc_cfg->BaseAddress);
    XAdcPs_SetSequencerMode(&xadc_ctl, XADCPS_SEQ_MODE_CONTINPASS);
    XAdcPs_SetSeqChEnables(&xadc_ctl, (1 << CH_VAUX14));

    xil_printf("Resetting Hardware...\n");
    XGpio_DiscreteWrite(&gpio_start, 1, 0);
    XGpio_DiscreteWrite(&gpio_rst_conv, 1, 1);
    XGpio_DiscreteWrite(&gpio_rst_fp,   1, 1);
    usleep(1000);
    XGpio_DiscreteWrite(&gpio_rst_conv, 1, 0);
```

```
    XGpio_DiscreteWrite(&gpio_rst_fp,   1, 0);
    xil_printf("System␣Ready.\n");

    while(1){
        u32 pot_raw, pot_val_12bit;
        u32 pwm_result;
        u32 done_status;

        pot_raw = XAdcPs_GetAdcData(&xadc_ctl, CH_VAUX14);
        pot_raw >>= LSB_BITS_NOT_USED;
        pot_val_12bit = pot_raw & 0xFFF;

        XGpio_DiscreteWrite(&gpio_pot_val, 1, pot_val_12bit);
        XGpio_DiscreteWrite(&gpio_start, 1, 1);

        do {
            done_status = XGpio_DiscreteRead(&gpio_done, 1);
        } while (done_status == 0);

        pwm_result = XGpio_DiscreteRead(&gpio_pwm_val, 1);
        XGpio_DiscreteWrite(&gpio_start, 1, 0);


        xil_printf("Pot:␣%4d␣|␣", pot_val_12bit);
        float pot_f = (float)pot_val_12bit;
        print_float_custom("Val_Float", pot_f);
        xil_printf("␣|␣PWM_U:␣%3d\r\n", pwm_result);

        usleep(50000);
    }

    cleanup_platform();
    return 0;
}
```

Anexa D – Diagrama de interfațare a componentelor sistemului

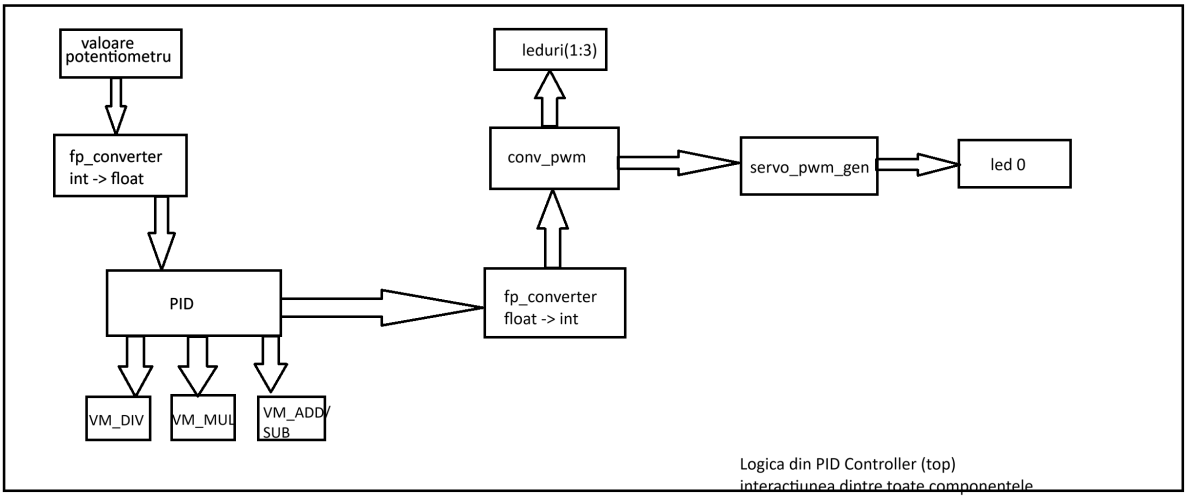

Figura 5: Logica din PID Controller (top) și interacțiunea dintre toate componentele